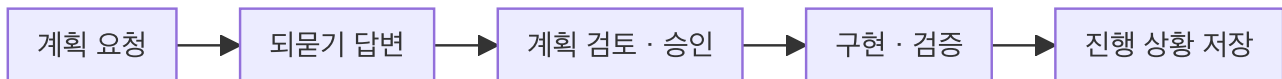


Claude Code: Plan Mode로 생각시키기

박사홍 · sahong.pak@gmail.com

Road map



- I. Claude Code 설치하기 — 설치 · 환경 변수 · 처음 실행 · 로그인 · 모델 선택
- II. 계획부터 세우기 — 계획 요청 · 되묻기 · 검토와 승인 · 계획서
- III. 계획대로 진행하기 — 구현 · 검증 · PLAN.md 저장 · 팀 프로젝트 활용

I. Claude Code 설치하기

설치하기

PowerShell에서 한 줄 명령으로 설치

```
irm https://claude.ai/install.ps1 | iex
```

🔗 설치 결과 확인

- Claude Code successfully installed! 와 버전 표시
- 설치 위치 `C:\Users\<사용자>\.local\bin\claude.exe`
- Setup notes 의 PATH 안내는 오류가 아니라 다음 단계(환경 변수 등록) 안내

⚠️ Path 등록 확인

새 PowerShell 창에서 `claude --help` 가 실행되지 않으면 Path 등록 필요

```
PS C:\Users\SSAFY\Desktop> irm https://claude.ai/install.ps1 | iex
Setting up Claude Code...

✔ Claude Code successfully installed!

Version: 2.1.258

Location: C:\Users\SSAFY\.local\bin\claude.exe

Next: Run claude --help to get started

⚠ Setup notes:
• Native installation exists but C:\Users\SSAFY\.local\bin is not in your PATH. Add it by opening:
  System Properties → Environment Variables → Edit User PATH → New → Add the path above. Then
  restart your terminal.

✔ Installation complete!
```

환경 변수 등록

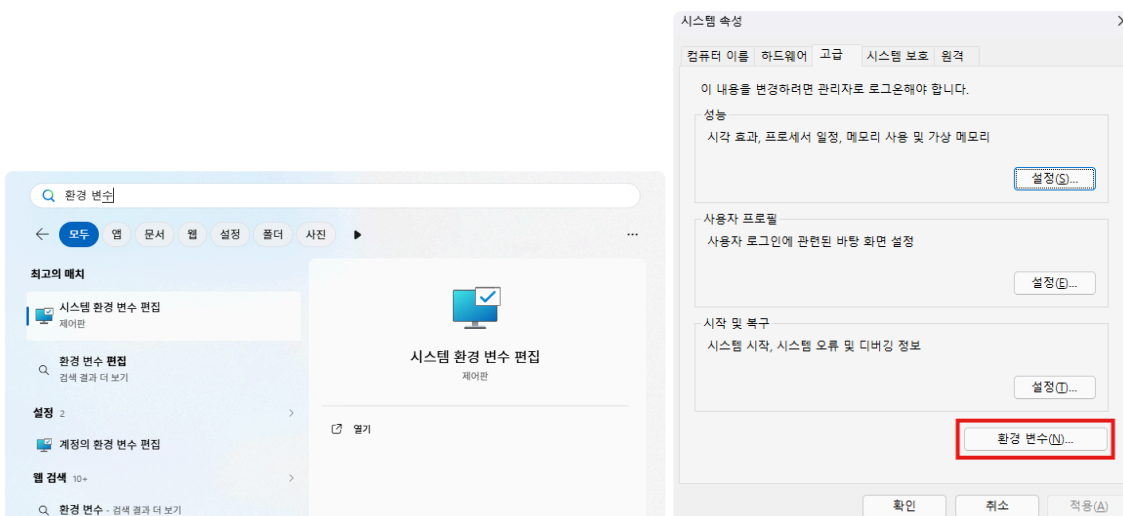
설치 경로를 Path에 추가해야 어느 폴더에서든 `claude` 명령 실행 가능

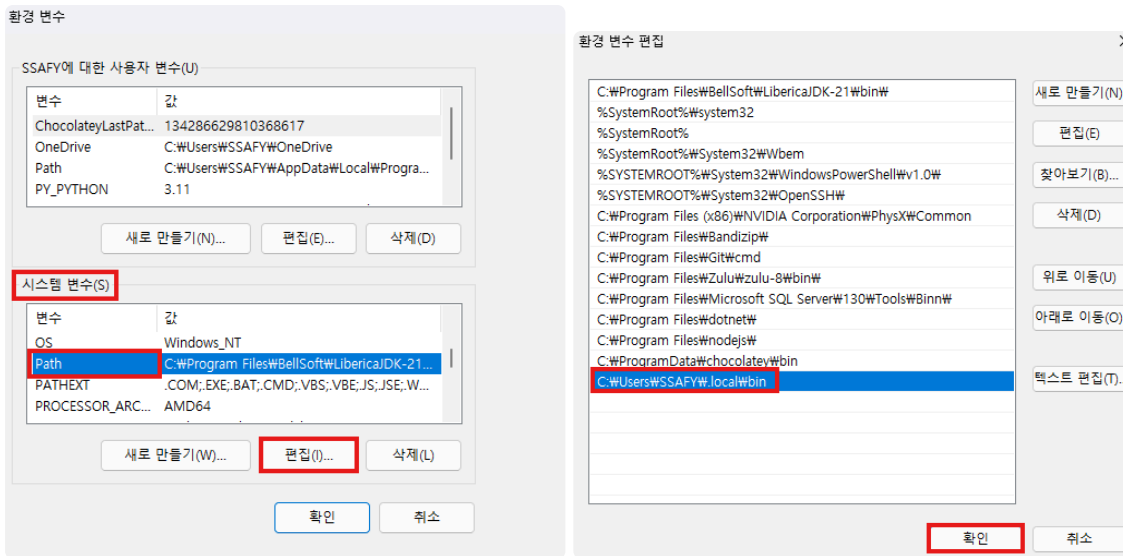
📖 순서

1. 검색창에 '환경 변수' 검색 → 시스템 환경 변수 편집
2. 시스템 속성 > 고급 탭 > 환경 변수(N)
3. 시스템 변수에서 **Path** 선택 → 편집(I)
4. 새로 만들기 → `C:\Users\<사용자>\.local\bin` 입력 → 확인

⚠ 새 창에서 확인

적용 후 PowerShell 창을 새로 열어야 변경된 Path 반영





처음 실행하기

PowerShell에서 `claude` 를 입력하면 테마 선택과 로그인 순서로 진행

① 순서

1. 테마 선택 (`/theme` 으로 변경 가능)
2. 로그인 방식 선택 → **1. Claude account with subscription**
3. 브라우저 로그인 → `Login successful` 확인 → Enter
4. 작업 폴더 접근 확인 → **Yes, I trust this folder**

② ‘이 폴더를 신뢰합니까?’ 는 무슨 뜻일까?

Claude Code가 이 폴더의 파일을 읽고 · 수정하고 · 실행할 수 있게 허용한다는 뜻
내가 만든 프로젝트나 잘 아는 오픈소스라면 Yes
출처가 불분명한 폴더라면 먼저 내용을 살펴본 뒤 결정

모델 선택

`/model` 로 상황에 맞는 모델 선택 (처음에는 Default 그대로 두어도 충분)

모델	이런 때
Opus 5 (Default)	일상적인 작업부터 복잡한 작업까지 기본값
Fable 5.1	가장 어렵고 오래 걸리는 작업
Sonnet 5	반복적인 일상 작업을 효율적으로
Haiku 4.5	빠른 답변이 필요할 때

- 프롬프트 창에 `/model` 입력 → 번호로 선택

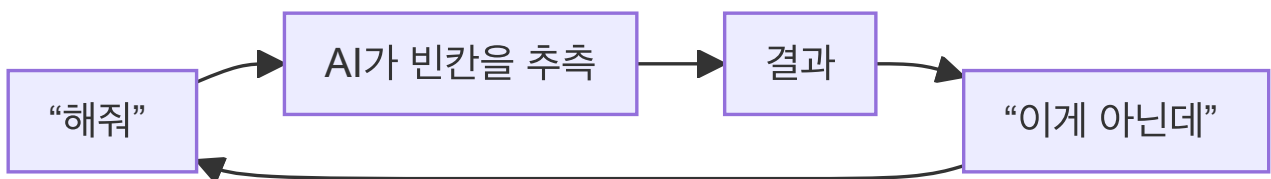
- 모델 이름 옆 설명으로 각 모델의 용도 확인

II. 계획부터 세우기

‘해줘’ 한 마디로는 왜 안 될까?

흔한 프롬프트의 모습과 결과가 한 번에 안 나오는 이유

- > Todo 앱 만들어줘
 - Vue? React? 순수 JS? → 정해진 게 없어 React로 진행
 - 저장은? → 새로고침하면 사라지는 메모리 저장
 - 기능은? → 추가 · 삭제만
- > 아니 Vue로. 그리고 새로고침해도 남아야지
 - 처음부터 다시 만드는 중...
- > 수정 기능도 있어야 하는데. 그리고 팀원이 읽을 설명도
 - 구조를 바꾸는 중... (세 번째)



한 바퀴마다 다시 만들기 — 3~4번 반복 = 시간 낭비 + 코드 꼬임

❓ 왜 한 번에 안 될까?

요청에 없는 결정(기술 · 저장 · 범위 · 독자)을 AI가 대신 내림 — 그 결정이 내 생각과 다르면 결과도 다름

사람에게도 마찬가지 — 신입에게 ‘로그인 해줘’만 말하면 되문거나 다르게 만들

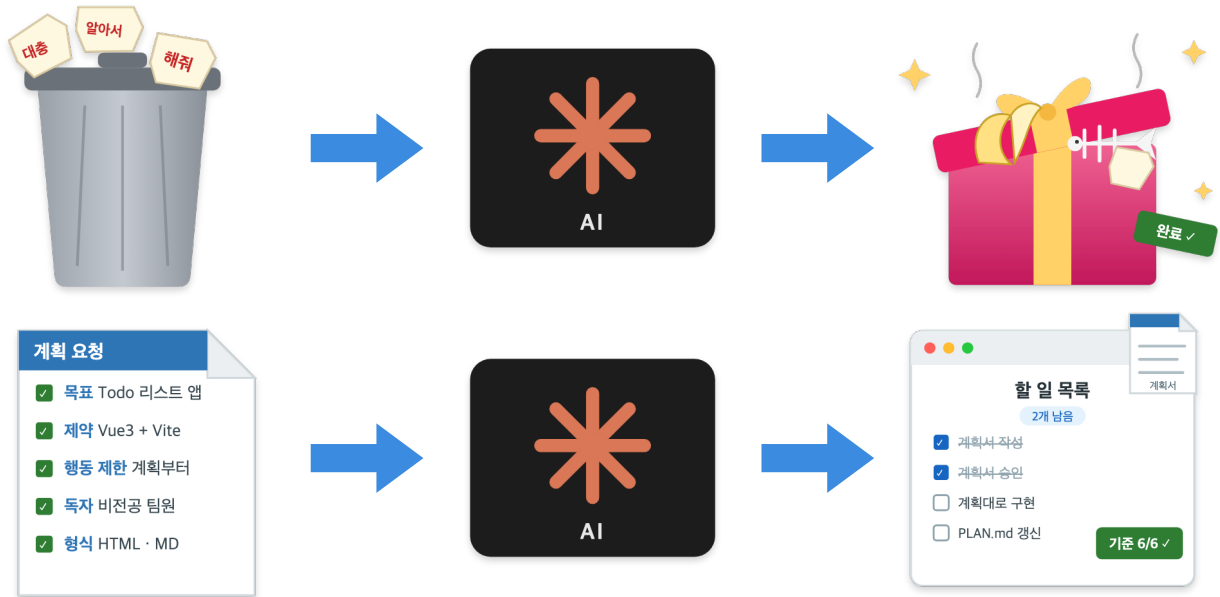
① 계획 = 빈칸 채우기

빈칸을 미리 채워 주는 것이 곧 계획 — 그래서 코드보다 계획서가 먼저

쓰레기를 넣으면 예쁜 쓰레기가 나옴

AI는 요청의 빈칸을 그럴듯하게 채움 — 겉은 멀쩡해도 내가 원하는 것이 아님

	입력	결과
대충 요청	‘해줘 · 대충 · 알아서’ → 빈칸을 AI가 추측	포장(코드 · 문서)은 그럴듯 — 내용은 내가 원하는 것이 아님 · ‘완료 ✓’까지 붙어 더 위험
제대로 된 요청	목표 · 제약 · 행동 제한 · 독자 · 형식을 채운 요청	되묻기 → 계획서 → 승인 → 원하던 결과 · 완료 기준 6/6으로 ‘다 됐다’를 확인



❓ 왜 ‘예쁜’ 쓰레기가 더 위험할까?

그렇듯해서 잘못을 늦게 발견 — 발견이 늦을수록 수정 비용 급증 (아래 비용 그래프)

계획 요청, 전과 후

같은 목표라도 다섯 가지를 채우면 되묻기 두 번에 계획서가 나옴

요소	전 — 한 마디	후 — 다섯 가지를 채운 요청
목표	Todo 앱 (어떤 Todo?)	Todo 리스트 앱 — 추가 · 수정 · 삭제 · 완료
제약	×	Vue3 + Vite
행동 제한	×	바로 만들지 말고 계획부터
독자	×	프로그래밍을 잘 모르는 팀원
형식	×	회사 계획서처럼 HTML · MD 파일로 저장

AI가 없던 시절에도 설계가 먼저

소프트웨어 공학은 50년 전부터 ‘코드 전에 설계’를 말해 옴

결함 발견 단계	요구 분석	설계	구현	테스트	운영
수정 상대 비용	×1	×5	×10	×20	×100

- 요구 단계에서 1이던 수정 비용이 운영 단계에서는 100 안팎 — Boehm, *Software Engineering Economics* (1981) 등, 수치는 자료마다 달라도 방향은 같음

연도	사람 · 사건	설계를 먼저 둔 이유
1970	Royce	폭포수 모델 — 요구 → 설계 → 구현 → 검증, 설계를 별도 단계로
1975	Brooks	맨먼스 미신 — 개념적 일관성(conceptual integrity)은 설계에서 나옴
1981	Boehm	결함 수정 비용 곡선 — 늦게 발견할수록 비쌘
2001	애자일 선언	계획을 없앴 것이 아니라 짧게 자주 계획 — 스프린트 계획 · 회고
—	SSAFY 프로젝트	기획서 · 와이어프레임 · ERD · API 명세가 곧 설계 문서

✎ Plan Mode는 새 습관이 아님

원래 하던 설계를 AI에게도 시키는 것 — 설계 문서를 AI가 대신 쓰고, 나는 검토와 승인

왜 계획이 먼저인가

인테리어 공사에 견적서와 도면이 먼저 필요한 이유와 같음

견적서 없이 바로 공사	견적서 · 도면부터 만들고 공사
타일 파손 · 누수 · 기울어진 가구 · 바닥재 혼용 · 문 틈새 불량	균일한 타일 · 통일된 바닥재 · 정돈된 가구 · 계약서 기준 하자보수

❓ 코드부터 짜면 왜 문제가 생길까?

중간에 방향이 흔들리고 빠뜨리는 기능 발생 — 계획서는 공사 전에 그리는 ‘설계 도면’

Plan Mode

코드를 건드리지 않고 ‘어떻게 만들지’만 먼저 정리하는 모드

📄 Plan Mode 켜기

`shift + tab` 으로 모드 전환 → 입력창 아래 `plan mode on` 표시

```
> Todo 리스트 앱을 만들고 싶어. Vue3와 Vite를 사용해야 해.
바로 만들지 않고, 어떻게 만들지 계획부터 세워줘.
프로그래밍을 잘 모르는 팀원도 읽을 수 있는 계획서로.
회사에서 사용하는 계획서처럼 HTML과 MD 파일로 저장해줘.
```

📄 plan mode on (shift+tab to cycle)

🔗 Plan Mode에서 Claude가 하는 일

- 파일을 읽고 탐색 — 코드 수정 없음
- 정해지지 않은 것을 먼저 되묻기
- 계획서를 쓰고 승인을 기다림 → 승인 후 실행

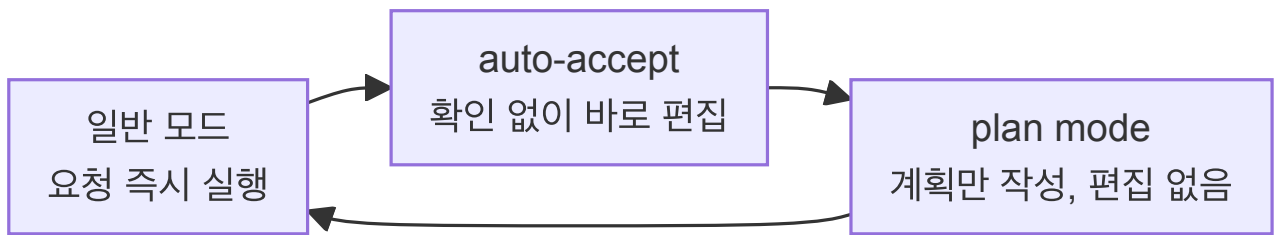
	일반 모드	Plan Mode
코드 수정	요청 즉시 수정	계획만 작성, 수정 없음
진행 방식	바로 실행	검토 · 승인 후 실행
어울리는 일	작은 수정, 오타 고치기	새 기능 · 큰 변경 · 계획서

⚠️ 모드 선택 주의

큰 변경을 일반 모드로 시작하면 되돌리기 어려움

Plan Mode 조작 한눈에

`shift + tab` 한 번이면 모드가 바뀌고, 승인 화면의 세 선택지가 다음 행동을 정함



선택지	뜻	이럴 때
1. Yes, and use auto mode	편집 확인 없이 끝까지 실행	계획을 충분히 읽었을 때
2. Yes, manually approve edits	파일 편집마다 확인	공용 코드 · 처음 쓰는 저장소
3. Tell Claude what to change	계획을 고치고 다시 검토	체크리스트에 x 가 있을 때

📄 관련 명령

`/plan` — 승인 전 계획 미리보기 · `Esc` — 진행 중단 · 승인 뒤에도 “계획서를 수정해줘”로 되돌아갈 수 있음

⚠️ 승인 선택지 주의

1번은 편하지만 되돌리기 어려움 — 팀 저장소에서는 2번이 기본

오늘 실습에서 누른 순서 — `shift + tab` → `plan mode on` → 계획 요청 → 되묻기 2개 답변 → `Ready to code?` → 1. Yes, and use auto mode (실습이라 1번, 팀 프로젝트에서는 체크리스트 뒤에 2번)

계획 요청

무엇을 만들지와 어떻게 만들지, 계획부터 세워 달라고 요청

> Todo 리스트 앱을 만들고 싶어. Vue3와 Vite를 사용해야해.
바로 만들지 않고, 어떻게 만들지 계획부터 세워줘.
프로그래밍을 잘 모르는 팀원도 읽을 수 있는 계획서로.
회사에서 사용하는 계획서처럼 HTML과 MD 파일로 저장해줘.

▮ `plan mode on (shift+tab to cycle)`

✎ 프롬프트에 담긴 다섯 가지

- 목표 — Todo 리스트 앱
- 제약 — Vue3와 Vite 사용
- 행동 제한 — 바로 만들지 말고 계획부터
- 독자 — 프로그래밍을 잘 모르는 팀원
- 형식 — 회사 계획서처럼 HTML · MD 파일로 저장

Claude의 되묻기

정해지지 않은 것을 먼저 질문 — 답에 따라 계획서의 분량과 저장 위치가 달라짐

← [] 기능 범위 [] 저장 위치 ✓ Submit →

- Todo 리스트 앱의 기능 범위를 어느 수준으로 잡을까요?
 - ↳ 1. 기본 기능만 (추천) — 추가/수정/삭제/완료, 브라우저 자동 저장
 - ↳ 2. 기본 + 분류/우선순위 — 카테고리 · 우선순위 · 마감일까지
 - ↳ 3. 잘 모르겠음, 알아서 정해줘
- 계획서(MD/HTML) 파일을 어디에 저장할까요?
 - ↳ 1. 02 폴더 루트 (추천) 2. resource/001_계획 요청 안
- Review your answers → 기본 기능만 · 02 폴더 루트 → Submit

❓ 왜 바로 계획서를 쓰지 않을까?

범위가 곧 분량과 기간
애매한 결정은 사람의 몫
추천 답이 있어 고르기만 해도 진행

✎ 선택지가 보여주는 것

범위를 넓히면 계획서가 길어짐
저장 위치는 이후 작업의 기준 폴더
Chat about this — 더 물어볼 수도 있음

되묻기에 답하는 요령

선택지 하나가 계획서의 분량 · 위치 · 범위를 정함 — 세 가지 답 방식

방식	언제	예시
추천 선택	잘 모르거나 데모 · 첫 시도일 때	“1. 기본 기능만 (추천)” → 계획서가 짧고 빠름
직접 입력 (Type something)	팀 규칙 · 기술 스택이 이미 정해져 있을 때	“Vue3 + Pinia, 저장은 팀 API 사용” → 계획서가 우리 프로젝트에 맞춤
대화 (Chat about this)	선택지가 모두 아니거나 판단할 정보가 부족할 때	“두 방식의 장단점을 먼저 설명해줘” → 결정 근거를 얻고 나서 선택

질문	답	달라지는 것
기능 범위	기본 기능만 / 기본 + 분류 · 우선순위	계획서 분량 · 개발 기간 · 완료 기준 개수
저장 위치	폴더 루트 / resource 안	이후 작업의 기준 폴더 · 팀원이 찾는 위치

❗ 되돌아가기

Review your answers 화면에서 되돌아가 고칠 수 있음 — Submit 전까지는 비용 0

계획 검토와 승인

Ready to code? — Claude가 계획을 보여주고 승인을 기다림

Here is Claude's plan:

- Todo 리스트 앱 개발 계획서 작성 계획
 - ↳ Context — 코드가 아니라 계획서 2개(MD · HTML)가 산출물
 - ↳ 사용자 확인 사항 — 기본 기능만 · 02 폴더 루트
 - ↳ 계획서 구성 — 개요 → 배경 → 용어 → 기능 범위 → 기술 → 로드맵 → 완료 기준 → 제약
 - ↳ 유의점 — 앱 코드는 만들지 않음 · 전문 용어마다 쉬운 말
 - ↳ 검증 방법 — MD 다시 읽기 · HTML 브라우저로 확인
- Would you like to proceed?
 - ↳ 1. Yes, and use auto mode 2. Yes, manually approve edits 3. Tell Claude what to change

항목	내용
맥락	왜 · 무엇을 만드는지
산출물	파일 2개, 저장 위치
구성	계획서 목차 8개
유의점	하지 말아야 할 것
검증	어떻게 확인할지

① 계획 수정 요청

3번으로 계획 자체를 수정 요청 가능 — 코드 전에 고치는 것이 가장 저렴

승인 전 검토 체크리스트

Ready to code? 에서 1번을 누르기 전에 확인할 다섯 문항

- ☐ 범위 — 이번에 만들 것과 만들지 않을 것이 모두 적혀 있는가
- ☐ 완료 기준 — ‘다 됐다’를 확인할 수 있는 문장인가 (예: 새로고침해도 목록 유지)
- ☐ 순서 — 단계가 의존 관계 순서로 놓였는가 (환경 → 뼈대 → 기능 → 저장 → 다듬기 → 점검)
- ☐ 독자 — 비전공 팀원이 읽을 수 있는 용어 설명이 있는가
- ☐ 검증 방법 — 계획 안에 ‘어떻게 확인할지’가 있는가

① 체크리스트에 x가 있으면

하나라도 비어 있으면 → 3. Tell Claude what to change 로 계획부터 수정

- > 완료 기준을 ‘새로고침해도 목록 유지’처럼 확인 가능한 문장으로 바꿔줘
- > 제외 항목을 표로 정리하고 ‘나중에 확장 가능’이라고 표시해줘

계획서 — 결과물

Todo_리스트_앱_개발_계획서.html · .md — 표지 · 목차 · 표 · 로드맵이 있는 보고서

#	장	내용
1	문서 개요	목적 · 작성일 · 대상 독자 · 문서 상태
2	배경과 목적	왜 만드는지, 인테리어 도면 비유
3	용어 설명	Vue3 · Vite · 컴포넌트 · 상태 · 로컬 스토리지
4	기능 범위	포함 7개 · 제외 4개
5	사용할 기술	Vue 3 · Vite · 로컬 스토리지 · 기본 CSS
6	개발 로드맵	6단계 타임라인
7	완료 기준	체크리스트 6개
8	참고 · 제약	로컬 저장 한계, 이후 확장 가능성

- 결과물: [계획서 HTML](#) · [계획서 MD](#)

용어 설명과 기능 범위

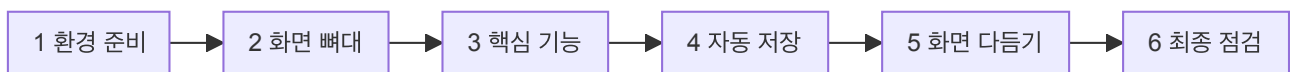
용어	쉬운 설명
Vue3	화면을 블록처럼 조립해 만드는 도구
Vite	저장하자마자 브라우저에 반영되는 빠른 작업실
컴포넌트	화면을 이루는 부품 — 입력창, 할 일 한 줄
상태(state)	앱이 지금 기억하는 정보 — 할 일 목록, 완료 여부
로컬 스토리지	브라우저 안의 저장 공간 — 새로고침해도 유지

- 포함하는 기능 7가지 — 추가 · 목록 보기 · 완료 체크 · 수정 · 삭제 · 남은 할 일 개수 표시 · 자동 저장 (로컬 스토리지)
- 이번 범위에서 제외 — 로그인 · 회원가입 / 여러 기기 간 동기화 / 서버 · 데이터베이스 저장 / 카테고리 · 우선순위 · 마감일

⚠ 제외 항목도 기록

제외 항목도 적어야 확장 후보로 남음

로드맵과 완료 기준



완료 기준 — 이 정도면 다 만든 것

- ☐ 할 일을 입력하고 등록하면 목록에 바로 표시
- ☐ 체크박스로 완료 표시(취소선), 다시 누르면 취소
- ☐ 목록에서 할 일 내용 수정 가능
- ☐ 목록에서 할 일 삭제 가능
- ☐ 화면에 '남은 할 일 개수' 표시
- ☐ 새로고침 · 브라우저 재시작 후에도 목록 유지

❓ 완료 기준은 왜 필요할까?

'다 됐다'를 사람과 Claude가 같은 기준으로 판단
그대로 검증 단계의 체크리스트가 됨

III. 계획대로 진행하기

계획서 기반 진행

- > 계획서를 기반으로 프로젝트를 진행해줘.
- Scaffold created. Installing dependencies now.

● Write(src\App.vue) Write(src\components\TodoInput.vue)

● Write(src\components\TodoList.vue) Write(src\components\TodoItem.vue)

● Write(src\style.css) Update(index.html) Update(README.md)

● Server is up at http://localhost:5173/ Called claude-in-chrome 3 times

● 계획서의 완료 기준 6개 항목을 모두 만족합니다.

계획서	1 환경 준비	2 화면 뼈대	3 핵심 기능	4 자동 저장	5 화면 다듬기	6 최종 점검
실제 진행	Vite로 Vue3 프로젝트 생성	App.vue에 입력창 + 목록	TodoInput · TodoList · TodoItem	localStorage watch로 연동	style.css 정리	브라우저에서 직접 확인

구현 결과

todo-app/ — Vue 3 + Vite, 외부 라이브러리 없이 기본 CSS

파일	역할
App.vue	할 일 상태 관리, localStorage 자동 저장
components/TodoInput.vue	입력창 — 추가
components/TodoList.vue	목록
components/TodoItem.vue	항목 — 완료 체크 · 인라인 수정 · 삭제
style.css	기본 스타일
README.md	실행 방법

```
npm install
npm run dev    →    http://localhost:5173/
```

검증

Claude가 브라우저를 직접 열어 완료 기준 6개를 하나씩 확인

#	완료 기준	확인 결과
1	등록하면 목록에 바로 표시	추가 즉시 반영, '2개 남음'으로 개수 갱신 ✓
2	완료 체크 → 취소선, 다시 누르면 취소	체크 시 취소선 표시 ✓
3	할 일 내용 수정 가능	'보고서 작성' → '보고서 최종 제출' ✓
4	할 일 삭제 가능	'우유 사기' 목록에서 제거 ✓
5	남은 할 일 개수 표시	헤더에 표시 ✓
6	새로고침 후에도 목록 유지	localStorage로 유지 ✓

🔗 계획서가 있어서 가능한 일

완료 기준이 곧 테스트 목록 — '다 됐다'를 같은 기준으로 판단

계획대로 진행해도 남는 찌꺼기

계획서가 있어도 검증 단계는 필요 — 실제 진행 기록에서 발견한 것

```
<script setup>
import HelloWorld from './components/HelloWorld.vue' // ← Vite 템플릿 찌꺼기
import { ref, computed, watch } from 'vue'
import TodoInput from './components/TodoInput.vue'
import TodoList from './components/TodoList.vue'
...
<template>
  <HelloWorld />
  <!-- ← 계획에 없던
컴포넌트 -->
  <div id="app">
```

🔗 왜 생겼나 → 어떻게 잡혔나 → 교훈

- 1단계(환경 준비)의 템플릿 파일을 지우기 전에 2단계(뼈대)의 App.vue를 먼저 작성 — 정리 순서 누락
- 같은 세션 뒤쪽 'cleanup of unused boilerplate files'에서 정리 — 최종 todo-app/src/App.vue에는 HelloWorld 없음
- 완료 기준 + 브라우저 검증이 없었다면 찌꺼기가 화면에 남은 채 '완료'로 끝났을 것

⚠️ 최종 파일 확인

진행 기록(중간 산출물)과 최종 파일은 다를 수 있음 — 끝나면 최종 파일을 직접 열어 확인

> 완료 기준 6개를 브라우저에서 하나씩 확인하고, 계획에 없는 파일이 남아 있으면 알려줘

진행 상황 저장

실행 결과를 PLAN.md로 남기고, 계획서처럼 HTML로도 변환

- Write(PLAN.md) Wrote 39 lines to PLAN.md
 - └ # Todo 리스트 앱 진행 계획
 - └ 원본 계획서: ../resource/001_계획 요청/...계획서.md
 - └ - [x] 1. 프로젝트 환경 준비 ... - [x] 6. 전체 점검

> PLAN.md도 계획서처럼 HTML로 만들어줘

- Write(PLAN.html) Wrote 250 lines to PLAN.html
- 계획서와 같은 스타일로 만들어 브라우저로 열어드렸습니다.

PLAN.md 구성

- 개발 진행 단계 6개 — 전부 ✓
- 완료 기준 6개 — 전부 ✓
- 추후 진행 가능한 기능 9개 — ☐ 로 보존

- 결과물: [PLAN.md](#) · [PLAN.html](#)

❓ 왜 대화 창에 두지 않고 파일로 저장할까?

- 다음 세션 · 팀원이 이어서 진행 — 원본 계획서 링크와 마지막 업데이트 날짜가 함께 기록
- 범위 밖 기능이 후보 목록으로 보존 — 이번에 뺀 기능이 사라지지 않고 다음 단계 후보로 남음
- 발표 · 보고 자료로 재사용 — PLAN.html 한 장이 곧 진행 보고서

추후에 진행할 수 있는 기능 9개 — 로그인 · 회원가입 / 여러 기기 간 동기화 / 서버 · DB 저장 / 카테고리 분류 / 우선순위 설정 / 마감일 설정 / 필터 · 보기 전환 / 검색 기능 / 드래그로 순서 변경

PLAN.md 루틴

세션을 시작하고 끝낼 때 두 프롬프트로 이어 가기

세션 시작

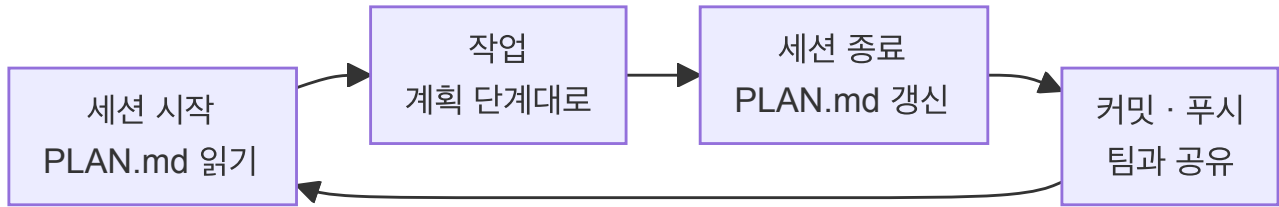
> PLAN.md 읽고 다음 단계부터 진행해줘

- PLAN.md 확인 — 6단계 중 4단계까지 완료
- 5단계 ‘화면 다듬기’부터 진행합니다

세션 종료

- > PLAN.md 갱신해줘 — 오늘 한 일 체크, 남은 일과 후보 기능 정리
- 5 · 6단계 완료 표시, 후보 기능 2개 추가
- 마지막 업데이트: 2026-09-05

응답 줄은 예시 — 실제 문구는 프로젝트 상태에 따라 달라짐



📌 PLAN.md 커밋

PLAN.md를 git에 커밋하면 팀원의 Claude 세션에서도 같은 진행 상황으로 이어짐

흔한 실수 5가지

실수	대처
계획을 읽지 않고 1번 승인	체크리스트 다섯 문항을 먼저 — 승인 뒤 고치면 코드까지 다시
첫 범위를 너무 넓게 잡음	‘기본 기능만’처럼 작게 시작 — 나머지는 후보 목록으로
완료 기준 없이 “다 됐어?”	확인 가능한 문장으로 완료 기준을 요구 — 그것이 곧 테스트 목록
계획서를 대화창에만 둬	MD · HTML로 저장하고 커밋 — 다음 세션과 팀원이 이어받음
오타 수정까지 Plan Mode	작은 수정은 일반 모드 — 새 기능 · 큰 변경 · 계획서만 Plan Mode

✍️ 기준 한 줄

새 기능 · 큰 변경 · 계획서 = Plan Mode / 오타 · 작은 수정 = 일반 모드 — `shift + tab` 한 번 이 기준

정리

Plan Mode 흐름 — 내가 하는 일과 Claude가 하는 일

단계	계획 요청	되묻기 답변	계획 검토 · 승인	구현 · 검증	진행 상황 저장
내 역할	요청에 목표 · 제약 · 독자 · 형식 담기	기능 범위와 저장 위치 선택	계획을 읽고 승인 또는 수정 요청	동작 결과 확인	PLAN.md · HTML 요청
Claude	파일 탐색, 정해지지 않은 것 파악	선택지 제시, 답을 기다림	계획서 작성 (코드 수정 없음)	6단계 순서대로 구현, 브라우저로 검증	진행 단계 · 완료 기준 · 후보 기능 기록

🔗 계획 요청에 넣을 것

목표 무엇을 만들지 · 제약 기술 · 조건 · 행동 제한 바로 만들지 말고 · 독자 누가 읽는지 · 형식 어떤 파일로

AI의 답변, 어떻게 검증할까?

할루시네이션은 결함이 아니라 특성 — 틀린 내용도 자신 있게 말하므로 검증은 사용자의 몫

	① 근거 요구 — AI에게	② 직접 재현 — 내가	③ 교차 확인 — 다른 눈
방법	기술 선택 · 단계의 근거 요구 / 존재하는 패키지 · API인지 확인 / 확신 없는 부분은 ‘추정’ 표시 요구	‘완료했습니다’를 믿지 않고 완료 기준을 직접 재현 / 최종 파일을 열어 diff 확인	PLAN.md · PR을 팀원이 리뷰 / 공식 문서와 대조 / 새 세션에 계획 검토 요청
예시	“계획에서 추정한 부분과 확인이 필요한 부분을 표시해줘”	브라우저에서 기준 6개 직접 클릭 — HelloWorld 잔재도 이렇게 발견	“이 계획서의 빠진 단계나 위험을 리뷰어 입장에서 찾아줘”

⚠ 위험 신호 — 이럴 때는 멈추고 검증

존재하지 않는 라이브러리 · 명령 · 근거 없는 ‘모두 만족’ · 진행 기록과 다른 최종 파일 · 질문 없는 매끄러운 계획

📄 검증 장치

Plan Mode의 승인 단계와 완료 기준이 곧 검증 장치 — 검증 비용은 잘못 믿은 비용보다 항상 작음

기획서 → 계획서 → PLAN.md

SSAFY에서 이미 쓰는 문서와 Claude 계획서의 관계 — 무엇을 · 어떻게 · 어디까지

	기획서 · 와이어프레임 · ERD · API	개발 계획서	PLAN.md · PLAN.html
뜻	무엇을 만들지	어떻게 만들지	어디까지 했는지
누가 · 무엇	팀이 작성 — 서비스 · 화면 · 데이터 · 인터페이스	Plan Mode가 작성, 팀이 승인 — 범위 · 기술 · 로드맵 · 완료 기준 · 제외	실행 후 Claude가 갱신 — 진행 단계 ✓ · 완료 기준 ✓ · 후보 기능 ☐
Plan Mode에서	이미 있는 문서를 그대로 입력	“기획서를 읽고 개발 계획서를 세워줘”	세션마다 읽고 갱신, git에 커밋

> docs/기획서.md 와 ERD.png 를 읽고, 바로 만들지 말고 개발 계획서부터 세워줘.
팀원 모두 읽을 수 있게 MD와 HTML로 저장해줘.

📄 기획서의 힘

문서가 이미 있으면 되묻기가 줄고 계획서가 우리 프로젝트에 맞춰짐 — 기획서가 곧 다섯 요소의 답

계획서, 누가 무엇을 읽나

같은 계획서를 역할마다 다른 장부터 — ‘프로그래밍을 잘 모르는 팀원도’라는 요청이 필요한 이유

역할	먼저 읽는 장	확인할 것
비개발 팀원 (기획 · 디자인)	2. 배경과 목적 · 3. 용어 설명 · 4. 기능 범위 · 7. 완료 기준	내가 기대한 기능이 범위에 있는지 / 제외 항목에 동의하는지
개발자 (프론트 · 백엔드)	5. 사용할 기술 · 6. 개발 로드맵 · 7. 완료 기준 · 8. 참고 · 제약	단계 순서가 의존 관계에 맞는지 / 완료 기준을 테스트로 옮길 수 있는지
팀장 · PM (일정 · 범위 관리)	1. 문서 개요 · 4. 제외 항목 · 6. 로드맵 · 8. 제약 · 후보 기능	일정 안에 가능한 범위인지 / 다음 단계 후보가 남아 있는지

> 이 계획서를 기획 담당 · 개발자 · 팀장 세 사람의 관점에서 각각 검토하고, 빠진 것을 알려줘

📄 체크리스트의 출처

역할별 확인 항목이 곧 승인 전 체크리스트의 출처 — 계획서 장 번호는 실습 계획서(8장) 기준

팀 프로젝트에서 활용하기

1학기 · 2학기 팀 프로젝트 모두 — 코드보다 계획서를 먼저 저장소에 남기는 습관

단계	상황	프롬프트 예시	Plan Mode 단계
프로젝트 시작	기획서 → 개발 계획서	“기획서를 읽고 개발 계획서를 세워줘. 바로 만들지 말고, 팀원 모두 읽을 수 있게” / “우리 팀 기술 스택으로 폴더 구조와 역할 분담안을 계획서로 정리해줘”	계획 요청
기능 개발	기능 단위로 계획 → 승인	“이 기능을 어떻게 구현할지 기존 코드 구조를 지켜서 계획부터 세워줘” / “승인하기 전에 영향 받는 파일 목록을 먼저 보여줘”	계획 검토 · 승인
통합 · 디버깅	수정 계획을 먼저 확인	“이 버그를 고치기 전에 원인과 수정 계획을 먼저 보여줘” / “리팩터링 범위와 순서를 계획서로 만들고 승인 후에 진행해줘”	계획 검토 · 승인
발표 · 문서	PLAN.md → 진행 보고서	“PLAN.md를 진행 보고서 HTML로 만들어줘. 심사위원도 읽을 수 있게” / “남은 기능 후보 목록을 다음 스프린트 계획으로 정리해줘”	진행 상황 저장

❗ 모드 선택 기준

새 기능 · 큰 변경은 Plan Mode 먼저 — 작은 수정은 일반 모드

⚠ 승인 전 읽기

승인 전에 계획을 끝까지 읽기 — 이때 고치는 것이 가장 저렴

계획서 · PLAN.md가 저장소에 있으면 팀원 누구나 같은 그림으로 이어받음

내 프로젝트로 써보기

다섯 칸을 채우면 계획 요청 프롬프트가 완성

요소	내 프로젝트
목표	무엇을 만들까 — 예: 팀 일정 공유 웹앱
제약	기술 · 조건 — 예: Spring Boot + Vue, 2주
행동 제한	바로 만들지 말고 계획부터
독자	누가 읽나 — 예: 기획 담당 팀원, 심사위원
형식	어떤 파일로 — 예: MD + HTML, 저장 위치

_____을 만들고 싶어. _____을 사용해야 해.
바로 만들지 않고, 어떻게 만들지 계획부터 세워줘.

_____도 읽을 수 있는 계획서로. _____ 파일로 저장해줘.

빈칸을 채운 문장을 그대로 Plan Mode에 입력 — 되묻기가 오면 답하고, 계획을 읽고, 승인

부록

프롬프트 모음 (복사해서 사용)

Todo 리스트 앱을 만들고 싶어. Vue3와 Vite를 사용해야해. 바로 만들지 않고, 어떻게 만들지 계획부터 세워줘. 프로그래밍을 잘 모르는 팀원도 읽을 수 있는 계획서로. 회사에서 사용하는 계획서처럼 HTML과 MD 파일로 저장해줘.

계획서를 기반으로 프로젝트를 진행해줘.

PLAN.md도 계획서처럼 HTML로 만들어줘

PLAN.md 읽고 다음 단계부터 진행해줘

PLAN.md 갱신해줘 — 오늘 한 일 체크, 남은 일과 후보 기능 정리

실습 자료

- resource/000_Claude Code Install/ — 설치 · 환경 변수 · 로그인 · 모델 선택 캡처
- resource/001_계획 요청/ — 인테리어 견적서 비유 SVG, Plan Mode 설정 캡처, 계획 요청 프롬프트와 되묻기 기록, 계획서 HTML
- resource/002_프로젝트 진행/ — 계획서 기반 진행 기록
- resource/003_계획서 저장/ — 계획서 MD, PLAN.md · PLAN.html
- resource/004_삽화/ — 쓰레기 투입 · 제대로 투입 SVG
- resource/todo-app/ — 완성된 Vue 3 + Vite Todo 앱 (`npm install` → `npm run dev`)